<table>
<tr><td colspan="2">Nombre: Emmanuel Carrillo Loera

Edgar Daniel del Ángel Hernández

Fatima Alejandra Cavazos Serrato

Equipo: 3</td><td>Matrícula: 2067988

2094613

2099488</td></tr>
<tr><td>Nombre del curso: Componentes de Lógica de Negocios</td><td colspan="2">Nombre del profesor: Miguel Ángel Gómez Marroquín</td></tr>
<tr><td>Modulo Netacad: 3</td><td colspan="2">Actividad: Reporte de actividad fundamental 3</td></tr>
<tr><td colspan="3">Fecha: 13/10/2026</td></tr>
<tr><td colspan="3">Bibliografía (Formato APA)

Archivo “Actividad_Unidad_3_CLN_ArquitecturaAlumnos.docx” proporcionado en la plataforma TEAMS

Archivo “03_Sazón_Urbano.docx” proporcionado en la plataforma TEAMS</td></tr>
</table>

Commands y Queries (CQRS básico)

Command = “haz que algo cambie”. Modifica el estado y aplica reglas.

Query = “dime cómo está algo”. Solo lee y nunca modifica nada.

Cada uno tiene su Handler, la clase que ejecuta esa operación.

No se separa la base de datos (hasta el código)

Los 5 métodos del Service Layer siguen siendo la puerta de entrada. Cada uno recibe un Command y se lo entrega a su Handler. Los pasos de coordinación son los mismos de la Etapa 4, solo que ahora viven en el Handler.

Tabla de Commands (cambian el estado)

Tipo	Nombre	Datos de entrada	Handler y dependencias	Resultado	Justificación
Command	`RegistrarPedido`	Canal, líneas, datos del cliente, mesa, responsable	`RegistrarPedidoHandler` → `PedidoRepository`, Unit of Work, publicador	Id, total y estado del pedido	Crea el pedido y aplica RN-01 y RN-14 (CU-01)
Command	`AplicarPromocion`	Id del pedido, id de la promoción, fecha-hora, responsable	`AplicarPromocionHandler` → `PedidoRepository`, `PromocionRepository`, `AplicadorDePromocion`, Unit of Work	Pedido con descuento y total recalculado	Cambia el total y exige una sola promoción (RN-04)
Command	`CancelarPedido`	Id del pedido, motivo, solicitante, autorizador (si aplica)	`CancelarPedidoHandler` → `PedidoRepository`, `MermaRepository`, Unit of Work, publicador	Cancelación confirmada, indicando si hubo merma	Aplica RN-02 y RN-03
Command	`MarcarProductoAgotado`	Producto o insumo, responsable	`MarcarProductoAgotadoHandler` → `DisponibilidadRepository`, notificador	Producto no disponible	Cambia la disponibilidad y avisa a Mesero y Caja
Command	`QuitarLineaAgotada`	Id del pedido, id de la línea, responsable	`QuitarLineaAgotadaHandler` → `PedidoRepository`, **`ConsultasDeDisponibilidad`**, Unit of Work	Pedido sin la línea y con total recalculado	Cancelación parcial (RN-06, RN-12)
Command	`AsignarRepartidor`	Id del pedido, id del repartidor, responsable	`AsignarRepartidorHandler` → `PedidoRepository`, `RepartidorRepository`, `EntregaRepository`, `AsignadorDeRepartidor`, Unit of Work	Entrega con repartidor asignado	Crea la entrega y marca al repartidor no disponible

Command	`RegistrarSalidaDeEntrega`	Id de la entrega, responsable	`RegistrarSalidaHandler` → `EntregaRepository`, Unit of Work	Entrega en ruta, con hora de salida	RN-11
Command	`ConfirmarEntrega`	Id de la entrega, responsable	`ConfirmarEntregaHandler` → `EntregaRepository`, `PedidoRepository`, `RepartidorRepository`, Unit of Work, publicador	Entrega completada y pedido entregado	RN-11 y cierre del pedido
Command	`RegistrarEntregaFallida`	Id de la entrega, motivo, responsable	`RegistrarEntregaFallidaHandler` → mismas dependencias que `ConfirmarEntrega`	Pedido en “entrega no completada”	RN-11: el motivo es obligatorio

Tabla de Queries (solo consultan)

Tipo	Nombre	Datos de entrada	Handler y dependencias	Resultado	Justificación
Query	`ConsultarResumenDelPedido`	Id del pedido	`ConsultarResumenHandler` → `ConsultasDePedidos`	`ResumenDePedidoDto`	Mesero y Caja ven el pedido antes de aplicar promoción o quitar una línea. Es el ejemplo del carrito que dio el profe
Query	`ConsultarMenuDisponible`	Fecha-hora	`ConsultarMenuHandler` → `ConsultasDeDisponibilidad`	Lista de `ProductoDisponibleDto`	Resuelve US-04: lo agotado “deja de mostrarse” sin meter una regla extra en `RegistrarPedido`
Query	`ConsultarPromocionesVigentes`	Fecha-hora	`ConsultarPromocionesHandler` → `ConsultasDePromociones`	Lista de `PromocionVigenteDto`	Caja elige la promoción de forma explícita (RN-05)
Query	`ConsultarRepartidoresDisponibles`	Ninguno	`ConsultarRepartidoresHandler` → `ConsultasDeRepartidores`	Lista de `RepartidorDisponibleDto`	El Encargado asigna uno disponible (US-14)

Los handlers de consulta dependen de interfaces de lectura (ConsultasDe…), no de los repositorios. Así no reciben métodos de escritura como guardar (principio I de SOLID).

Modelo de lectura (DTO/View Model)

DTA= Data Transfer Object. Es un objeto simple que solo transporta datos de una capa a otra, sin reglas ni métodos de negocio. Un View Model es un DTO armado justo para mostrarse en pantalla.

En nuestro diseño, es lo que devuelven las Queries (por ejemplo, ResumenDePedidoDto). Por eso no usamos el aggregate Pedido para mostrar datos: el aggregate protege reglas y el DTO solo muestra información.

DTO	**Campos**	**Por qué no es el modelo de escritura**
`ResumenDePedidoDto`	Id, canal, mesa o cliente (texto), líneas (producto, cantidad, modificaciones, subtotal), promoción (nombre y descuento), total, estado (texto), responsable (nombre)	Es plano y está listo para mostrar: no tiene métodos ni reglas, ni expone la preparación por estación ni las cancelaciones internas. El aggregate `Pedido` protege reglas; este DTO solo muestra datos
`ProductoDisponibleDto`	Id, nombre, precio, disponible (sí/no)	El catálogo no es un aggregate del dominio (decisión 3 de la Etapa 3)
`PromocionVigenteDto`	Id, nombre, productos, horario (texto)	Muestra la promoción sin su lógica de vigencia
`RepartidorDisponibleDto`	Id, nombre	Solo lo necesario para elegir

Hallazgos

#	Principio	Evidencia	impacto	Cambio realizado	Tipo de cambio
H1	S	Etapa 4: DisponibilidadApplicationService y EntregaApplicationService están separados de PedidoApplicationService	Ningún servicio hace “de todo”; marcar un producto agotado no se mezcla con la gestión de pedidos	Ninguno	Confirma buena decisión
H2	S	gestionarEntrega y gestionarProductoAgotado reciben varias solicitudes y podrían crecer	Si tuvieran la lógica adentro serían servicios “que hacen de todo”	Ya aplicado en la Etapa 6: el servicio solo enruta y cada paso vive en su Handler	Riesgo mitigado
H3	O	Etapas 3 y 4: no estaba definido quién calcula el descuento de una promoción. Si lo hiciera un servicio o Pedido, cada tipo nuevo (2x1, porcentaje) sumaría condiciones	Agregar un tipo de promoción obligaría a editar código que ya funciona	Promoción calcula su propio descuento. Un tipo nuevo se agrega como variante de Promoción, sin tocar Pedido ni el servicio	Corrección
H4	L	Etapa 5: el repositorio en memoria y el relacional podrían comportarse distinto (uno devuelve vacío y otra lanza error si no existe el pedido)	Cambiar de implementación alteraría el comportamiento del sistema	Contrato explícito: obtenerPorId siempre lanza un error de negocio (PedidoNoEncontrado, etc.) si no existe, en cualquier implementación	Correción
H5	I	Etapa 5: DisponibilidadRepository tenía estaAgotado y marcarAgotado, y QuitarLineaAgotadaHandler solo usa la consulta	Un consumidor depende de un método de escritura que no necesita	Se separa: DisponibilidadRepository conserva solo marcarAgotado, y estaAgotado pasa a la interfaz de lectura ConsultasDeDisponibilidad	Corrección
H6	I	Etapa 6: los handlers de consulta dependen de `ConsultasDe…` y no de los repositorios	Las consultas no reciben métodos como guardar	Ninguno	Confirma una buena decisión
H7	D	Etapas 4 a 6: dominio y handlers solo conocen interfaces; las implementaciones (SQL o memoria) están en infraestructura	Migrar de SQL Server a PostgreSQL toca solo infraestructura	Ninguno	Confirma una buena desición